Arcade

Harkouk Yanis - Legrand Eva

The compilation process:

We have a Makefile at the root of the project which builds the entire project. expect if you apply a rule next to it :

- The core Rule : builds the “./arcade” executable
- The games Rule : builds the games libraries in ./games
- The graphicals Rule : build the graphical libraries in ./display
- The all Rule : build the aforementioned rules

Usage:

To launch our Arcade, you will need to call the executable and a graphic library, found in the ./lib directory, with which you want to start the program with. It goes like this :

./arcade lib/arcade_*.so

Key to Use:

- M : goes back to the menu
- P : goes back to the previous graphical library
- N : goes to the next graphical library
- G : switches in between the available game
- BACKSPACE : quit the program
- Arrows : the arrows allow you to move in game
- R : restart the game
- ENTER : used to select the chosen library, graphical or games

Score:

Each time you play a game, the program looks for a file named “.name_of_the_game.scores”. If it does not exist, it creates it and writes the score in when the game is ended.

The file is filled as such : NAME: SCORE

Enums:

The two following enums are used in our IGameModule and IDisplayModule Interfaces. Their role is to facilitate the transfer and the retrieving of all data surrounding colors and user inputs. It allows us to use descriptive words instead of lengthy strings.

The Color Enum:

```
enum Color
   {
    DEFAULT,
    BLACK,
    WHITE,
    RED,
    BLUE,
    GREEN,
    YELLOW,
    GRAY,
    CYAN
};
```

The KeyPressed Enum:

```
enum KeyPressed
{
    OTHER,
    UP,
    DOWN,
    LEFT,
    RIGHT,
    R,
    P,
    N,
    G,
    M,
    BACKSPACE,
    CLOSE_BUTTON,
    ENTER
};
```

The GameStatus Enum:

This enum is only used in the IGameModule interface. Its purpose is to track the status of the game and handle it in our core accordingly.

```
    enum GameStatus
    {
        PLAYING,
        WON,
        WON_AND_RELOADED,
        LOOSE,
        QUIT,
        NO_ACTIVITY,
        MENU,
        RESET
    };
}
```

IGameModule Interface:

```
class IGameModule
   {
       public:
           IGameModule() = default;
           ~IGameModule() = default;

           virtual void resetGame() = 0;
           virtual std::string getGameName() const = 0;
           virtual std::size_t getScore() const = 0;
           virtual std::array<std::string, 23> getMap() const = 0;
           virtual std::size_t getMapCellSize() const = 0;
           virtual std::unordered_map<std::string, Color> getPattern()
const = 0;
           virtual std::vector<std::pair<std::string, std::pair<int,
int>>> getText() const = 0;
           virtual void incrementTime() = 0;
           virtual GameStatus updateMap(KeyPressed key) = 0;
           virtual std::pair<std::size_t, std::size_t> getIndexes()
const = 0;
           virtual void initText(const std::vector<std::string>, const
std::vector<std::string>) = 0;
   };
```

Our two games, Pacman and Nibbler, inherit from the IGameModule Interface which is composed of the following methods

- resetGame : reset to their beginning states all the variables of the class

- getPattern() : returns an unordered_map composed of an std::string which represents a pattern present in the game map and a Color value from the Color enum.
- incrementTime() : increments an std::size_t variable which represents the seconds that pass.
- updateMap(`KeyPressed key`) : update the game map based on the last user input (defined in the KeyPressed enum).
- getIndexes() : returns two ints associated with the vectors containing respectively the game libraries and the display libraries.
- initText(`const std::vector<std::string>, const std::vector<std::string>`) : initialize a vector of string (to send it to the core and then the display)

IDisplayModule interface:

```
class IDisplayModule
    {
        public:
            IDisplayModule() = default;
            ~IDisplayModule() = default;

            virtual void init(const std::unordered_map<std::string,
Color> pattern) = 0;
            virtual void stop() = 0;
            virtual void handleEvent(const std::string gameName) = 0;
            virtual void changeColor(const
std::unordered_map<std::string, Color> patternMap) = 0;
            virtual void displayText(const
std::vector<std::pair<std::string, std::pair<int, int>>> textToDisplay)
= 0;
            virtual void displayMap(const std::array<std::string, 23>
map, const std::size_t size) = 0;
            virtual KeyPressed getKey() const = 0;
            virtual void setKey(const KeyPressed newValue) = 0;
            virtual std::string getPlayerName() const = 0;
    };
```

Our three libraries, SFML / Ncurses / SDL2, inherit from the IDisplayModule Interface which is composed of the following methods

- init(`const std::unordered_map<std::string, Color> pattern)` : Initializes all the resources that will be used throughout the execution of the display. It fills a map composed of the pattern retrieved through our map and its corresponding object to display.
- stop() : liberate the resources used by the display.
- handleEvent(`const std::string gameName`) : this function handles the event happening while the display is running.
- displayText(`const std::vector<std::pair<std::string, std::pair<int, int>>> textToDisplay`) : retrieve the text to display from a map containing the string and its desired position and prints it.
- displayMap(`const std::array<std::string, 23> map, const std::size_t size`) : displays the map given in the parameter according to an internal variable initialized in the init() function.